Homework 2

CS 521: Fall 2025

Due on Friday, October 3, 2025 11:59 PM Central

Homework Policy: You are allowed to collaborate with your classmates, but report this in your submission. Even if you work on problems together, each student must write up their solution individually in their own words in their submission. This CS 473 course page on academic integrity is a handy reference: `https://courses.engr.illinois.edu/cs473/sp2023/integrity.html`. We have ZERO tolerance for breaches of academic integrity, such as, but not restricted to, plagiarism, use of LLMs except for when stated explicitly, etc.
You can take upto 3 days extra in total across the semester without explanation. Once you exhaust the three-day late submission allowance, any further late homework submissions will result in a 25% penalty per each extra day. If the submission is late by 5 minutes or more then the submission will be counted as 1 day late. If you have exceptional personal circumstances that will prevent you from submitting the assignment on time, write to the course staff as soon as possible. Unless explicitly stated, use of LLMs for solving homework or cheating from others is not allowed – we will detect it! There will be an oral **exam** if we detect violations, during which you will be asked to answer a randomly selected question from the homework. If you are unable to show sufficient knowledge, then you will get zero for the full homework.
Please format your submission as a PDF (file naming convention: *CS521_⟨netid⟩_hw2.pdf*) and upload it on Gradescope by the deadline. Try to start every problem on a new page, to accurately map pages to questions on Gradescope.
Programming Problems: Describe your solution at a high-level (if applicable). DO NOT paste tall walls of code into your submission! Upload your solution files to GitHub publicly and provide a link to the relevant files as part of your solution.

Points: Problem 1: 4 + 8 = 12 points, Problem 2: 8 + 12 = 20 points, Problem 3: 8 + 16 = 24 points, Problem 4: 24 points, Problem 5: 6 + 2 + 6 + 6 = 20 points; Total: 100 points.
There are two bonus questions, each worth 25 points. You can solve either one, or both of them, you will atmost get 25 points for the bonus section.

Problem 1 (Operations on Complex Intervals)**.** A complex interval is defined as the set of all complex numbers whose real and imaginary parts lie within given intervals:

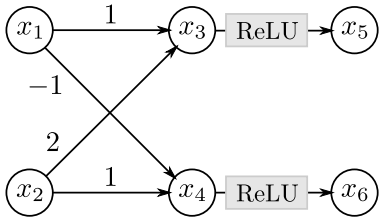
$$I = [a, b] + i[c, d] = \{\, x + iy \;\mid\; x \in [a, b],\; y \in [c, d] \}.$$

Consider two complex intervals: $I_1 = [a_1, b_1] + i[c_1, d_1], \quad I_2 = [a_2, b_2] + i[c_2, d_2]$.

(a) Compute the most precise complex interval soundly capturing the addition $I_1 + I_2$.

(b) Repeat the above for $I_1 \cdot I_2$.

Problem 2 (Certification using Zonotopes)**.** Consider the following small neural network with two input neurons x_1, x_2 and two output neurons x_5, x_6. The network consists of an affine layer followed by a ReLU layer.



$$x_5 = \mathrm{ReLU}(x_1 + 2x_2)$$
$$x_6 = \mathrm{ReLU}(x_2 - x_1)$$

You are given the following zonotope ϕ over the input neurons:

$$\phi: \quad \begin{pmatrix} \hat{x}_1 \\ \hat{x}_2 \end{pmatrix} = \begin{pmatrix} 2 \\ 1 \end{pmatrix} \cdot \varepsilon_1 + \begin{pmatrix} 1 \\ 2 \end{pmatrix} \cdot \varepsilon_2 + \begin{pmatrix} 4 \\ 3 \end{pmatrix}$$

Your goal is to prove that $x_5 \geq x_6$ for all inputs x_1, x_2 in the zonotope ϕ.

(a) Draw the 2D shape of ϕ.

(b) Is it possible to design Zonotope transformers for the affine and ReLU layers to prove the property? If yes, then show them. If not, then why?

Problem 3 (MILP encoding for Maxpool)**.** In the lecture, we learned the Mixed Integer Linear Programming (MILP) based encoding of the ReLU operation. In this exercise, your goal is to design an encoding of the Maxpool operation using MILP.

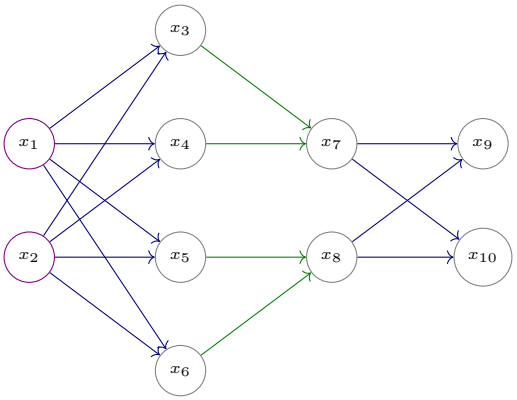


Figure 1: Fully-connected network with Affine and ReLU operations.

(a) Design a MILP encoding for the Maxpool operation $y := max(x_1, x_2)$ where the input bounds for x_1 and x_2 are $[a_1, b_1]$ and $[a_2, b_2]$ with $a_1, a_2, b_1, b_2 \in \mathbb{R}$.

(b) Now, consider the neural network shown in Fig. 1. The neural network has two input (x_1, x_2) and two output (x_9, x_{10}) neurons and consist of two layers with affine transformations (edges colored blue) and one layer with maxpool operation (edges colored green). The transformations in the network are given as:

$$
\begin{aligned}
x_3 &:= x_1 + x_2 \\
x_4 &:= x_1 - 2 \\
x_5 &:= x_1 - x_2 \\
x_6 &:= x_2 \\
x_7 &:= max(x_3, x_4) \\
x_8 &:= max(x_5, x_6) \\
x_9 &:= x_7 \\
x_{10} &:= -x_7 + x_8 - 0.5
\end{aligned}
$$

Use the MILP encoding for ReLU and Maxpool for verifying the property that for all values of $x_1, x_2 \in [0, 1]$, the output at $x_9 > x_{10}$. Can the MILP analysis prove this property? Show your work.

Problem 4 (Programming)**.** In this problem, you will implement interval analysis for a simple neural network. The starter code of this problem is at `https://github.com/nalinwadhwa02/CCS521FA26HW/blob/main/hw2/q4/mnist_interval_analysis.ipynb`.

Network Training. We provide a fully connected neural network. You need to train your network on MNIST with 15 epochs and report the clean accuracy.

Property. Let f be the network, and let x be an image with the label y. The network is locally robust at x for a radius ϵ if $\text{argmax}_j f_j(x') = y$ for all $x' \in [0,1]^{784}$ with $\|x' - x\|_\infty \leq \epsilon$. To verify this property, add an output layer that calculates $f_y(x') - f_j(x')$ for each $j \neq y$. Interval analysis proves the property if the lower bound of each of these outputs is not negative.

Task. Implement interval analysis for your network. Use this to measure the robustness for the whole test set of your network for 10 l_∞ neighborhoods, sized evenly between 0.01 and 0.1 (inclusive). Report the verified accuracy (the percentage of the images that are successfully verified) for those different values chosen. As usual, present your observations clearly along with experimental evidence (numbers, images, etc.) in your submission.

Solution Requirements. You should present your solution for this in the form of a Jupyter notebook. We recommend using Google Colab since we can interact with your solution easily, but you can also just upload the notebook to your GitHub repo.

Hint. You may refer to this repo `https://github.com/Zinoex/bound_propagation` for the implementation of interval analysis.

Problem 5 (Certifier Design in ConstraintFlow)**.** ConstraintFlow [SSMS24] is a DSL for DNN certifiers that use abstract interpretation. A certifier in ConstraintFlow has three parts:

- The abstract *shape* of each neuron, and the constraints that the shape must satisfy.
- The abstract *transformer* for each operation.
- The *flow* of the analysis through the network.

A certifier is *sound* if its abstract elements always contain the real values. That is, for each neuron n and each input in the input region, the real value of n is at least l_n and at most u_n. The same is true for the polyhedral bounds: $L_n \leq n \leq U_n$. A sound certifier can never prove a property that is false. A certifier is *more precise* if its bounds are tighter. A more precise certifier can prove a property for more images.

ConstraintFlow can automatically prove that a certifier is sound (ProveSound [SSMS25]). It can also compile the certifier to Python code and run it on a network. The ConstraintFlow code is at `https://github.com/uiuc-focal-lab/constraintflow`.

Starter code. The starter code is at `https://github.com/nalinwadhwa02/CCS521FA26HW/tree/main/hw2/q5`. It contains these files:

- `deeppoly.cf`: the DeepPoly certifier. Shape's in deeppoly are defined by two concrete bounds (l, u) and two polyhedral bounds (L, U). The Affine transformer calculates l and u by backsubstitution towthe input layer. The ReLU transformer uses the DeepPoly relaxation. If the input of a ReLU crosses zero, `relu_lower` selects the lower bound x or 0 with the area heuristic.
- `mnist_relu_3_100.onnx`: a fully connected MNIST network with three hidden layers of 100 ReLU neurons. The test accuracy of the network is 97.25%.
- `run_cf.py`: a script that compiles a certifier and runs it on the network.
- `README.md`: the setup instructions and examples. Read this file before you start.

Task. Your tasks are as follows:

(a) Design a new certifier that is *different* from DeepPoly. For example, you can change the abstract shape, the Affine and ReLU transformers, or the flow. Write the certifier in ConstraintFlow. Give the shape declaration and the Affine and ReLU transformers. Tell what is new in your certifier, and why you think it is useful.

(b) Use the `provesound` command of ConstraintFlow to check that your certifier is sound. Give the output. If the check fails, find the cause and correct your certifier.

(c) Use `run_cf.py` to run DeepPoly and your certifier on the given network. Use the first 100 MNIST test images, with $\epsilon \in \{0.005, 0.01, 0.02, 0.03, 0.05\}$. Give the percentage of verified images and the time for each image in a table. Compare your certifier against Deeppoly, if the results are the same, tell why. Put your `.cf` file on GitHub and give the link.

(d) Write a transformer for the HardTanh activation, $\text{HardTanh}(x) = \max(-1, \min(1, x))$, in the DeepPoly shape. In ConstraintFlow, the rule for this activation is named `HardTanh`, as in `HardTanh -> ...`. You can keep the Affine transformer from `deeppoly.cf`. Use `provesound` to check that your transformer is sound, and give the output. Note that `provesound` cannot check activations that are not piecewise linear, such as Sigmoid and Tanh.

Hint. DeepPoly keeps one polyhedral lower bound for each neuron. If the input of a ReLU crosses zero, two lower bounds are sound: 0 and x. DeepPoly selects one of them with a heuristic. For part (a), you can keep *both* lower bounds for each ReLU, and design a backsubstitution strategy that uses them.

Bonus Research Questions

Bonus Research Question 1 (25 points). You can solve this bonus question, the next bonus question, or both. The maximum total for the homework is 125 points. In this problem, we ask you to compute bounds on the worst-case error function for a VAE model with two inputs x_1 and x_2, when the same noise v with $||v||_\infty \leq \epsilon, \epsilon \in \mathbb{R}$ is added to them. The bound should hold with at least 95% probability. The worst-case error with respect to a given noise v added to x_i refers to the reconstruction loss (mean-squared error) between the original input $x_i + v$ to the encoder and the point on the corresponding output distribution at the decoder maximizing the error. You can consider a ReLU-based encoder and ReLU-based decoder, such as the one available at `https://github.com/uiuc-focal-lab/civet/tree/main`. You can also refer to the paper here [XBVS25] and [LXS+23], which is a real-world example of studying UAP on a VAE model. To adapt to the VAE model, you need to find the support set for the same noise added to those two inputs and bound the worst-case error of the decoder using relational verifications by designing a difference tracker.

Bonus Research Question 2 (25 points). LLMs are very large, and their input space has no simple structure. Thus, the deterministic verifiers from this course do not scale to LLMs. Recent work certifies LLMs *statistically*. In this approach, the specification has two parts: a *distribution* of prompts, and a property that the responses must satisfy. The certificate is a high-confidence bound on the probability that the LLM satisfies the property for a prompt from the distribution. Two examples are:

- LLMCert-B [CHK+25] (`https://openreview.net/pdf?id=HQHnhVQznF`) certifies counterfactual bias. It samples sets of prompts that are different only in a demographic attribute. Then it bounds the probability that the responses of the LLM are not biased.

- Lumos [CJP+25] (`https://arxiv.org/pdf/2512.02966`) is a DSL to write these specifications. The paper gives more examples: knowledge comprehension, multi-turn safety, privacy, and driving scenarios for vision-language models.

In this problem, you design your own specification and test it:

(a) **Describe a task.** Select a property of LLM behavior. For example: consistency under paraphrases, refusal of harmful requests, correct multi-step arithmetic, or privacy of information in the context. Your task must be different from the examples in [CHK+25, CJP+25].

(b) **Define the task formally.** Give these three items:

(i) The distribution $\mathcal{D}$ of prompts. Tell exactly how you sample one prompt, or

one set of related prompts. For example, you can sample from a graph, from templates, or from a dataset. The distribution must cover a large number of realistic prompts. A distribution with a few fixed prompts is not acceptable.

(ii) A function ψ that shows if a response (or a set of responses) satisfies the property.

(iii) The quantity to certify: $p = \Pr_{P \sim \mathcal{D}}[\psi(\mathcal{L}(P)) = 1]$ for an LLM $\mathcal{L}$.

(c) **Statistical certification.** Describe a method that calculates an interval $[p_l, p_u]$ from n i.i.d. samples. The interval must contain p with a probability of at least $1 - \delta$. For example, use Clopper-Pearson confidence intervals. Show why the guarantee is true. Tell how n and δ change the width of the interval.

(d) **Test on LLMs.** Implement your specification. Certify two or more LLMs with $\delta = 0.05$. You can use open-source models from Huggingface, or API-based models. Give the bounds. Tell what the bounds show about the models. Include examples of failures.

Note: You can use LLMs to make prompts or to judge responses in your specification. You must not use LLMs to write your solution. If you use LLMs in your specification, tell how this changes the guarantee.

References

[CHK+25] Isha Chaudhary, Qian Hu, Manoj Kumar, Morteza Ziyadi, Rahul Gupta, and Gagandeep Singh. Certifying counterfactual bias in LLMs. In *International Conference on Learning Representations (ICLR)*, 2025.

[CJP+25] Isha Chaudhary, Vedaant Jain, Prineet Parhar, Kavya Sachdeva, Avaljot Singh, Sayan Ranu, and Gagandeep Singh. Lumos: Let there be language model system certification, 2025.

[LXS+23] Zikun Liu, Changming Xu, Emerson Sie, Gagandeep Singh, and Deepak Vasisht. Exploring practical vulnerabilities of machine learning-based wireless systems. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 1801–1817, Boston, MA, April 2023. USENIX Association.

[SSMS24] Avaljot Singh, Yasmin Sarita, Charith Mendis, and Gagandeep Singh. Con-

straintflow: A dsl for specification and verification of neural network analyses. In *Static Analysis Symposium (SAS)*, 2024.

[SSMS25] Avaljot Singh, Yasmin Sarita, Charith Mendis, and Gagandeep Singh. Automated verification of soundness of dnn certifiers. In *Proc. ACM Program. Lang. (OOPSLA)*, 2025.

[XBVS25] Changming Xu, Debangshu Banerjee, Deepak Vasisht, and Gagandeep Singh. Support is all you need for certified VAE training. In *The Thirteenth International Conference on Learning Representations*, 2025.